DEVELOPER GUIDE · V0.11.0

Golden Thread

Persistent memory across Claude Code sessions — install, operate, verify, and maintain the vault.

September 2026 · Claude Code plugin · Requires Python 3.8+

CONTENTS

01 **The core idea**

- Why Claude forgets — and how the vault doesn't
- The promotion ladder
- The Core-rule tier
- Loading is lazy, by design

02 **Installation**

- Install from GitHub or zip
- What the installer does

03 **First-time setup**

- /gt:gt-init — initialize the vault
- /gt:gt-create — scaffold a project
- /gt:gt-ingest — import an existing project

04 **Vault layout**

- Top-level directories
- Project folder anatomy
- Source topology

05 **Daily workflow**

- /gt:gt-open — start a session
- /gt:gt-route — the middle of a session
- /gt:gt-work — end a session
- /gt:gt-query — ask the vault a question

06 **Context & verification**

— /gt-farm:gt-farm — route work to external AI

— /gt:gt-validate — verify claims by re-deriving

07 **Knowledge management**

— /gt:gt-promote — graduate a finding

— /gt:gt-refresh — update sources

— Where does this fact go?

08 **Maintenance**

— /gt:gt-lint — health checks

— /gt:gt-settings — automatic behaviour

— /gt:gt-runbook-lint and /gt:gt-review

— /gt-watch:gt-watch — upstream repo watch

— /gt-demo:gt-demo — guided tour

— /gt-flow:gt-flow — flow view (module)

09 **Knowledge wiki (gt-wiki)**

10 **Script reference**

11 **Troubleshooting**

1. The core idea

Claude's context window resets after every session. Golden Thread moves what you learn *out of the session* and into a structured vault on disk — before the session ends. The vault is an Obsidian folder tracked by git. The right slice of it is loaded at the start of every session.

The promotion ladder

Every fact lives at the **narrowest scope that is honest** and is promoted upward only when a second project proves it general.

```
session conversation
↓ /gt:gt-work
Projects/<slug>/memory/*.md ← this project, this detail
↓ /gt:gt-promote
research.md · decisions.md ← settled findings and choices
↓ /gt:gt-promote
Knowledge/ ← true for several projects
↓ /gt:gt-promote
global-memory/ ← true in every session, everywhere
↓ /gt:gt-promote (with hook wiring)
core-rules/ ← enforced via hooks, not merely stored
```

Facts move **up** as they prove general. They never move back down. Retiring a fact sets `status: stale` and records what superseded it — the file stays on disk.

The Core-rule tier

Above `global-memory/` sits the Core-rule tier. These are rules enforced via hooks wired into `~/.claude/settings.json` (`UserPromptSubmit` + `Stop` hooks) — they are re-asserted every session, not just stored as files. **`/gt:gt-init`** installs these hooks automatically.

> **STORED ≠ ENFORCED**
>
> A rule in `core-rules/` with no wired hook is inert — it looks correct and does nothing. **`/gt:gt-lint`** reports this as `core-unenforced`. Treat it as a real defect; never suppress it.

Loading is lazy, by design

`/gt:gt-open` reads the project's core docs and then reads `memory/MEMORY.md` **only** — one line per file. Memory files load on demand. A project with 70 notes costs ~80 lines to open instead of ~2,000. The one-line description in `MEMORY.md` is the entire basis for deciding whether a file is worth loading.

> THE DISCIPLINE
>
> Promote a fact only when a **second project** proves it general. When in doubt, file it narrow — promotion is cheap, demotion isn't.

2. Installation

Requirements: Python 3.8+, Claude Code installed and running.

Option A — Clone from GitHub

```
git clone <github repo>/golden-thread.git
cd golden-thread/golden-thread-plugin
bash install.sh
```

Option B — Install from zip

```
unzip golden-thread-plugin.zip
cd golden-thread-plugin
bash install.sh
```

> IMPORTANT
>
> **Restart Claude Code after install.** The `/gt:` skills register at startup only and will not appear until you do.

What the installer does

`gt` — memory, projects, session workflow, enforcement hooks — is installed into Claude Code's plugin cache, together with each **module** that is on.

MODULES

A module is an optional plugin in the same marketplace, versioned with gt and declared by a `module.json`. 0.15.0 ships six:

- `wiki` (`gt-wiki` , on) — knowledge base with immutable sources
- `demo` (`gt-demo` , on) — the guided tour
- `watch` (`gt-watch` , on) — upstream repo watch and its session-start report
- `report-card` (`gt-report-card` , on) — the session report card and close-out question; hooks only, no command

- **`flow`** (`gt-flow`, on) — offline HTML view of the event stream
- **`farm`** (`gt-farm`, off for a fresh install, on for upgraders) — work packets for an external AI

Choose with `bash install.sh --list-modules`, `--without <name>` and `--with <name>`; the choice is remembered. A module that is off leaves nothing behind, and its hooks are wired only while it is on.

```
~/.claude/plugins/cache/golden-thread-plugin/gt/0.15.0/
  .claude-plugin/   ← plugin metadata
  hooks/            ← enforcement hook scripts
  scripts/          ← vault_init.py, gt_lint.py, gt_settings.py, …
  skills/           ← /gt:* skill definitions (16 skills)
  templates/        ← vault scaffold templates
```

Updating

```
# after git pull or downloading a new zip:
bash install.sh
```

Each version installs into its own subdirectory; newer versions do not conflict with older ones.

Uninstalling

```
rm -rf ~/.claude/plugins/cache/golden-thread-plugin
```

Your vault and `~/.claude/vault-config.json` are unaffected.

3. First-time setup

`/gt:gt-init` — Initialize the vault

Run once per machine. Claude will ask:

1. **"Do you already have an Obsidian vault on this machine?"**
   - Yes → provide the path. Golden Thread adds only missing files; existing notes are untouched.
   - No → choose a path (default: `~/Documents/Obsidian/GoldenThreadVault`). Obsidian is optional — the vault is plain Markdown and works without it.
2. "What's your domain or team name?" — e.g. "Platform Engineering" or "My Team"

The script writes `~/.claude/vault-config.json`, scaffolds the vault, and **installs Core-rule enforcement hooks** into `~/.claude/settings.json`. Idempotent — safe to re-run.

For a vault that predates the Core-rule tier:

```
python3 $SCRIPTS/vault_init.py install-core-rules --vault ~/my-vault
```

> NOTE
>
> `CLAUDE.md` is read at session start. Rules added mid-session apply from the *next* session — that is not a bug; restart to pick them up.

`/gt:gt-create` — Scaffold a new project

INPUT	NOTES
Slug	Must match the folder name; lowercase, hyphens only
Title	Human-readable name
Domain	Coarse grouping, e.g. "platform", "data"
Topology	Where the code lives — see §4
Sub-project parent	Optional; slug of the parent project
Runbook	Whether to scaffold `runbook.md`

After scaffolding, Claude fills `idea.md` from your description. **`idea.md` is immutable once written** — the traceable "why".

`/gt:gt-ingest` — Import an existing project

Copies notes and memory files into the vault. Never moves or deletes originals.

> **WIKILINK TRAP**
>
> Renaming files destroys `[[wikilink]]` graphs. Measure first:
>
> ```
> grep -roh '\[\[[^]]*\]\]' --include='*.md' . | wc -l
> ```
>
> Prefer keeping filenames. Never repoint links inside `Sources/`.

4. Vault layout

```
<vault>/
  CLAUDE.md              Conventions and page schema
  index.md               Navigational index of all Knowledge pages
  log.md                 Append-only audit trail
  review-queue.md        Items flagged for owner review
  lint-declines.md       Suppressions with reasons

  Sources/               IMMUTABLE — never modified after creation
  Knowledge/             Cross-project wiki pages
  global-memory/         Loaded into every session
    MEMORY.md            Index — what actually gets read at session start

  Projects/
    CONVENTIONS.md       Stages, properties, domain taxonomy
    PROTOCOL.md          Cross-project process rules
    INFRASTRUCTURE.md    Server fleet — defined ONCE, linked from projects

    <slug>/
      README.md          Status board + YAML frontmatter
      source.md          Where the code lives; deploy topology
      idea.md            Origin story — IMMUTABLE
      research.md        Append-only dated findings
      decisions.md       Append-only numbered ADRs
      design.md          Current architecture — rewritten in place
      spec.md            Handoff artifact (when design complete)
      runbook.md         Operational procedures (optional)
      memory/
        MEMORY.md        Index — the ONLY file gt-open reads
        <topic>.md       Loaded on demand
```

Project folder anatomy

FILE	MUTABILITY	PURPOSE
`idea.md`	Immutable	The "why" — never changed after creation
`source.md`	Updated	Code location, deploy topology, file plan
`research.md`	Append-only	Dated findings and gotchas
`decisions.md`	Append-only	Numbered ADRs
`design.md`	Rewritten	Current architecture — always describes NOW
`spec.md`	Created once	Handoff artifact with acceptance criteria
`runbook.md`	Append / update	Operational procedures and landmines
`memory/*.md`	Updated per session	Session state; carries level/enforcement frontmatter

Source topology

TOPOLOGY	MEANING
`local`	Folder on this machine. No deploy step.
`remote`	One repo, one server. Server pulls from GitHub.
`bastion-jump`	Several servers via one gateway host.
`bastion-direct`	Several servers, each independently reachable.

Bastion topologies carry a **file plan**: each deployed file is marked *static* (byte-identical everywhere) or *unique* (per-server; never cross-deployed). The fleet is defined once in `Projects/INFRASTRUCTURE.md` — never copied into a project file.

5. Daily workflow

```
/gt:gt-open my-project   # load context, see where you left off

# ... work happens ...

/gt:gt-route             # drifted? where does this belong, and am I in the right

/gt:gt-work              # write back findings before closing
```

`/gt:gt-open <slug>` — Start a session

Reads CONVENTIONS.md and PROTOCOL.md once per session, then the project in order:

```
source.md → idea.md → research.md → decisions.md
          → design.md → spec.md → runbook.md
          → INFRASTRUCTURE.md (bastion topologies)
          → memory/MEMORY.md   # index only; files load on demand
```

If `research.md` exceeds ~200 lines, reads headings first and only loads targeted sections. Announces the review-queue item count if non-zero. Summarises stage, topology, hosts, next action, blockers, and which memory files exist but are unloaded.

`/gt:gt-route` — The middle of a session

`gt-open` runs before the session knows what it is; `gt-work` runs after it is over. Between them sits the part where the work actually changes shape, and nothing served it.

It reads only the conversation already in context — never project files, because reading `research.md` to decide where a note goes costs more than the note — and answers four things: what this has become (naming the drift explicitly), where the output belongs, whether the project, harness and model are right, and one next action. It writes nothing except a single `INBOX.md` line.

"Nothing, keep going" is a designed-for answer. A check that always finds work is a check nobody runs twice. Reach for it when a session opened as one thing and became another, when a finding may belong to a project other than the one loaded, or when you cannot name what you are doing. Being wrong costs one re-run.

`/gt:gt-work` — End a session

The step most people skip — and skipping it is what causes the vault to decay.

DESTINATION	WHAT BELONGS THERE
`research.md`	New findings, dated. New discoveries only.
`decisions.md`	Numbered ADR with context and rejected alternatives. Stable choices only.
`design.md`	Rewritten in place when architecture changed. Describes *now*.
`runbook.md`	Operational commands and known landmines.
`memory/*.md`	Session state, updated in place.
`spec.md`	Created when design is complete enough to hand off.

Memory files carry frontmatter to make durability explicit:

```
metadata:
  type: core | feedback | user | reference
  level: core | context | generic      # default: generic
  enforcement: validated | reminder    # required only for level: core
```

Default to `generic`. Do **not** set `level: core` in `gt-work` — Core is a promotion that requires wiring a hook, which is `gt-promote`'s job.

After writing, `gt-work` asks whether any finding applies beyond this project and should be promoted. It also updates `stage:`, `topology:`, and `domain:` in `README.md` frontmatter when the project's reality changes.

`/gt:gt-query` — Ask the vault a question

Reads `index.md`, follows wikilinks 2–3 hops into `Knowledge/`, falls back to grep. Be specific. If a result shows `status: stale`, verify it before acting.

6. Context & verification

`/gt-farm:gt-farm` — Route work to external AI

Routes bulk, mechanical, or second-opinion tasks *out* of this context to an external AI service. The unit is a **work packet**: a self-contained Markdown document with a strict `FINDINGS` / `GAPS` return contract. Transport is swappable — paste into a web UI today; a script sends it to an API later. The packet is identical either way.

All four gates must pass before a task leaves. Any failure means the work stays here.

GATE	QUESTION
Stateless	Answerable with no vault or repo state?
Self-contained	Fits in a paragraph plus a list of URLs?
Checkable	Verifiable without redoing the work?
Releasable	Every input safe to hand a third party?

> **RELEASABLE IS DEFAULT-DENY**
>
> Never include: account identifiers, balances, hostnames, internal IPs, credential locations, non-public source code, or personal identifiers. Check this gate while *building* the packet, not before sending it.

Key rules:

- One packet, one task. A packet asking three questions returns three half-answers.
- Supply a vocabulary block with exact model IDs and product names read from the system — never from memory. A stale prior produces a stale query.
- Results come back as `unverified`. Promote only after opening the `source:` URL and confirming the claim.
- `GAPS` is mandatory; `GAPS: None` must be justified, not asserted.
- Cite the specific page carrying the claim — never a homepage or section index.

Packets are saved to `<vault>/Projects/external-ai-tools/packets/YYYY-MM-DD-slug.md` for auditability.

`/gt:gt-validate` — Verify claims by re-deriving

Verifies a claim by **re-deriving it**, not by reviewing the reasoning that produced it. Give a validator your reasoning and it grades the argument — and inherits the same blind spot. Give it only the claim, the rules, and the artifact, and it must go back to primary sources.

Use before recording a finding as fact, before a production change, or when a number matters.

VALIDATOR CLASSES

CLASS	USE WHEN
`empirical`	A number, measurement, or result could be wrong
`vantage`	The measurement position may not be able to observe the answer
`rule-compliance`	Work must satisfy `core-rules/` or `CONVENTIONS.md`
`code`	Code may not do what its name, comment, or docs claim

Build the packet with exactly three fields: `claim`, `rules`, `artifact`. Strip all conclusions, confidence signals, and adjectives that imply the expected answer. If it says "confirm that X", rewrite it as "determine whether X".

Outcomes: **confirmed** / **refuted** / **cannot-verify**. `cannot-verify` is never a pass — name what was missing. Append the verdict to `research.md`. A refutation supersedes the original finding.

7. Knowledge management

`/gt:gt-promote` — Graduate a finding

The judgement call: **has a second project proved this general?**

FROM → TO	WHEN
`memory/` → `research` / `decisions`	Finding stopped changing session to session
project → `Knowledge/`	A second project hit the same truth
`Knowledge/` → `global-memory/`	Needed in every session, every project
any → Core tier	Rule must be enforced, not just stored — `gt-promote` wires the hook

Nothing is deleted. Retiring sets `status: stale` and records the superseding fact.

`/gt:gt-refresh` — Update sources

Checks `Sources/` for upstream changes. Creates a new immutable file with `supersedes:` — never edits the old one. Knowledge pages citing the old source are flagged for update.

Where does this fact go?

THE FACT	GOES TO
"This preset is wrong right now"	`memory/`
"This API rejects plural keys"	`research.md`
"We use POST for all mutations, and why"	`decisions.md`
"The system is structured like this now"	`design.md`
"This platform behaviour bit two projects"	`Knowledge/`
"Always diff before overwriting prod"	`global-memory/`
"How to restart the prod backtest"	`runbook.md`
"Never scp without backing up"	`PROTOCOL.md`
"This code lives on host X at path Y"	`source.md`
"Which box serves which role"	`INFRASTRUCTURE.md`

TWO FAILURE MODES

Too narrow: a platform truth buried in one project — the next project rediscovers it the hard way.
Too wide: a quirk in `global-memory/` loading into unrelated sessions as a rule that isn't true there.

When unsure, file narrow. Promotion is cheap; demotion isn't.

8. Maintenance

`/gt:gt-lint` — Health checks

CHECK	WHAT IT CATCHES
`index-gap`	Knowledge page not in `index.md` — `gt-query` won't find it
`broken-link`	`[[wikilink]]` resolving to nothing
`orphan`	Knowledge page linked from nowhere
`memory-unlisted`	`memory/*.md` not in `MEMORY.md` — Claude will never load it
`global-gap`	`global-memory/*.md` not in its index
`memory-bloat`	`global-memory/` file over 30 lines
`global-scope-leak`	Project-specific content in `global-memory/`
`superseded-cited`	Knowledge page citing a superseded source
`stale`	Knowledge page marked `status: stale`
`source-todo`	`source.md` with no topology or deployment targets
`frontmatter`	Project README missing properties, or slug ≠ folder name
`core-misplaced`	A rule declares `level: core` but lives outside `core-rules/`
`core-no-enforcement`	`level: core` with no `enforcement` field declared
`core-unenforced`	The declared enforcement hook is not actually wired

> **NEVER SUPPRESS CORE-UNENFORCED**
>
> This is the machine-checkable form of "rule stored but never asserted". Suppressing it recreates the original bug with a paper trail saying it was fine.
>
> Fix: `vault_init.py install-core-rules --vault <vault>`

TRIAGE WORKFLOW

1. **You broke it** — fix now
2. **Already broken** — record in `review-queue.md`; don't guess at a target
3. **False positive** — suppress in `lint-declines.md` with the reason

`/gt:gt-settings` — Automatic behaviour

View and toggle everything Golden Thread does without being asked.

SETTING	VALUES	DEFAULT	WHAT IT DOES
`component_updates`	`off` · `report` · `confirm` · `auto`	`report`	At session start, compares installed hooks/scripts against plugin source and reports drift
`report_card`	`off` · `minimal` · `full`	`minimal`	At `/compact`, summarises session hygiene
`watch`	`off` · `report`	`off`	Hourly repo-watch fetch and the session-start report
`install_demo`	`yes` · `no`	`yes`	Whether `install.sh` installs `/gt-demo:gt-demo`, its script and the PizzaBot template

```
python3 $SCRIPTS/gt_settings.py show            # current state
python3 $SCRIPTS/gt_settings.py explain <name>  # reasoning behind the default
python3 $SCRIPTS/gt_settings.py set <name> <value>
```

`/gt:gt-runbook-lint` and `/gt:gt-review`

gt-runbook-lint finds procedures duplicated across runbooks and routes them to the right shared layer: `PROTOCOL.md`, a Knowledge page, or a repo `CLAUDE.md`.

gt-review scans Obsidian daily notes for uncaptured tasks and ideas, then promotes selected ones into tracked projects.

`/gt-watch:gt-watch` — Upstream repo watch

A watch is a vault note (`Projects/golden-thread/watches/<slug>.md`) naming a git URL and what to care about. `gt_watch.py fetch`, run from cron without Claude, asks the remote what changed (`git ls-remote` plus a shallow fetch; `gh` only for releases and advisories), classifies each change as **p0**, **review** or **routine** by fixed rules, and appends it to `~/.claude/golden-thread/watch/events.jsonl`. A SessionStart hook reads that queue — never the network — and

reports unacknowledged changes, P0 first. Each state file has one writer: the fetch owns `state.json` and `events.jsonl`, the skill owns only the `acked.json` cursor. Setting `watch`, default `off`.

P0 means: a new security advisory; a tag or release naming a CVE/GHSA id or announcing a security fix; a commit carrying a CVE/GHSA id; a `SECURITY.md` or watched-path change saying the same; or a watch's own `p0_when` pattern. Bare words like "auth" never are.

`/gt-demo:gt-demo` — Guided tour

Module `demo` (plugin `gt-demo`). A tour on PizzaBot 3000 — nine core acts plus one from each installed module that ships one (wiki, watch, flow) — run in its own throwaway vault at `~/.claude/golden-thread/demo-vault`. `start` builds it and prints the pinned launch command (`GT_VAULT=<demo vault> GT_WATCH=report GT_WATCH_STATE=… claude`); in that session `tour` performs each act with the real skill and waits on a Next button; `end` lists what the tour produced; `clean` rebuilds the vault; `remove` deletes it; `bash install.sh --without demo` removes the module.

`/gt-flow:gt-flow` — Flow view

Module `flow` (plugin `gt-flow`), new in 0.15.0. Renders the vault's `events.jsonl` as one self-contained HTML file that opens offline: a lane per project, time left to right, an arrow each time an item climbed a level. Add `--redact` before the page leaves your screen; task events are hidden until clicked or `--tasks`. A vault older than the events recovers its history with `gt_events.py --vault <vault> backfill --dry-run`.

The acts live in `templates/demo-pizzabot/tour.md`, one section each with `narration:`, `do:` and `point:` lines. The demo never writes `vault-config.json` — every skill and hook honors `$GT_VAULT` — and `clean` / `remove` delete only a directory carrying the demo's marker file.

9. Knowledge wiki (gt-wiki)

The `gt-wiki` plugin adds a structured knowledge base with immutable sources. Every ingested document is stored verbatim in `Sources/` and never modified. Knowledge pages are synthesized summaries that link back to the originals.

Setup and usage

```
/gt:gt-wiki-init                          # initialize
/gt:gt-wiki-ingest <url or file path>     # add a source
/gt:gt-wiki                               # query
/gt:gt-wiki-lint                          # health check
/gt:gt-wiki-refresh                       # check sources for upstream changes
```

Deterministic helpers

Two stdlib scripts in `golden-thread-wiki/<ver>/scripts/` take the mechanical parts out of the LLM's hands:

- **`wiki_log.py`** — the only writer of `log.md` and `index.md` entries. `log` appends `## [YYYY-MM-DD] <op> | <title>` from a closed vocabulary (ingest, query, lint, refresh, graduate, retire, relocate); `index` adds a page entry under a section, or replaces an existing one in place.
- **`wiki_refresh.py`** — report-only change detection. For a source with `local:` it runs `git fetch` and `git diff base..head`, where *base* is the source's `upstream_sha:` or the last commit on or before its `ingested:` date. Web-only sources are listed as `fetch-needed`. Exit 0 = nothing changed, 1 = changes found, 2 = error.

Page format

```
---
title: Page Title
category: concept | tool | runbook | decision | reference
tags:
  - lowercase-hyphenated
sources:
  - "[[Sources/filename.md]]"
created: YYYY-MM-DD
updated: YYYY-MM-DD
status: seed | growing | mature | stale
---
```

> **CROSS-LINKS ARE THE NAVIGATION LAYER**
>
> Claude follows `[[wikilinks]]` hop by hop from `index.md`. Link bidirectionally. Cross-domain links (A ↔ B from different areas) are the most commonly missed and the most valuable for multi-hop questions.

10. Script reference

```
SCRIPTS=~/.claude/plugins/cache/golden-thread-plugin/gt/0.11.0/scripts
```

vault_init.py

```
# Fresh vault
python3 $SCRIPTS/vault_init.py fresh \
  --vault ~/my-vault --domain "My Team"

# Create a project
python3 $SCRIPTS/vault_init.py create-project \
  --vault ~/my-vault \
  --name my-project --title "My Project" \
  --tags "backend,api" --domain platform \
  --topology bastion-direct --runbook

# Create a sub-project
python3 $SCRIPTS/vault_init.py create-project \
  --vault ~/my-vault --name sub-feature \
  --parent my-project --domain platform

# Connect an existing vault to a new machine
python3 $SCRIPTS/vault_init.py connect --vault ~/existing-vault

# Install Core-rule enforcement hooks (for older vaults)
python3 $SCRIPTS/vault_init.py install-core-rules --vault ~/my-vault
```

gt_ingest.py

```
python3 $SCRIPTS/gt_ingest.py ~/Projects/my-project --json
```

gt_lint.py

```
python3 $SCRIPTS/gt_lint.py ~/my-vault \
  --queue ~/my-vault/review-queue.md
```

gt_settings.py

```
python3 $SCRIPTS/gt_settings.py show
python3 $SCRIPTS/gt_settings.py explain component_updates
python3 $SCRIPTS/gt_settings.py set report_card off
```

11. Troubleshooting

SYMPTOM	CAUSE & FIX
"vault-config.json not found"	Run `/gt:gt-init`
Skills not showing after install	Restart Claude Code. Skills register at startup only.
A rule in `CLAUDE.md` is ignored	Read at session start only. Rules added mid-session apply next session. Run `/hooks` or restart.
Memory files aren't loading	By design — `gt-open` loads on demand. Ask for the file by name.
A memory file is never offered	Run `/gt:gt-lint` — likely `memory-unlisted`.
`gt-query` can't find a page you added	Likely `index-gap`. Add the page to `index.md`.
Core rules apply inconsistently	Run `/gt:gt-lint` and look for `core-unenforced`. Fix: `vault_init.py install-core-rules --vault <vault>`
Timestamp missing from responses	Enforcement hook unwired. Run `/hooks` or re-run `install.sh`.
Want to turn off the session report	Run `/gt:gt-settings set report_card off`
Lint suppression isn't working	Paths are matched case-insensitively. Use the path exactly as lint reports it.
"conflict: vault-config.json points to a different vault"	Run `vault_init.py connect --vault <new-path>`